SUMMARY: ADVANCED RAG TOPICS

Objective

By the end of this session students will be able to:

- Explain why **traditional IR metrics** (Recall@K, Precision@K) are not enough for evaluating RAG pipelines.
- Define the role of **RAGAS evaluation** in measuring end-to-end system quality, not just retrieval quality.
- Differentiate between **offline metrics** (Context Recall, Context Precision, Answer Correctness) and **online metrics** (Faithfulness, Answer Relevancy, Context Relevancy, Context Utilization).
- Implement **RAGAS evaluations in code** using Gemini (or other LLMs) as the evaluation judge.
- Interpret metric scores to diagnose **where failures occur** (retriever, context prep, or generation).
- Apply **improvement strategies** when metrics are weak (e.g., tune `top_k`, add reranker, compress context, tighten prompts, lower temperature).
- Design **reliability fallbacks** for empty-hit retrievals, stale indexes, and unsafe generations.
- Use **guardrails** (prompt-based, validation chains, structured parsers) to reduce hallucinations and enforce safe outputs.
- Build a **feedback loop**: run pipeline → evaluate with RAGAS → identify weak points → apply fixes → re-evaluate.
- Contrast a static **RAG skeleton** (fixed steps, fixed knobs) with a dynamic **RAG system** (adaptive retrieval, corrective loops, guardrails).

Introduction to RAGAS Evaluation

When you think about the RAG pipelines you've already built, you've touched every dial a builder can touch. You know how to rerank candidates with a cross-encoder when retrieval feels noisy. You know how to use multi-query expansion when one phrasing isn't enough. You know how to deduplicate context so the LLM doesn't waste half its input window repeating the same passage. You've even seen adaptive query strategies, switching retrieval depth depending on

whether the question is broad (“compare turmeric vs saffron coffee”) or narrow (“ratio of milk to coffee in beaten coffee”).

That toolbox is full. But here’s the catch: a toolbox only matters if you know when to reach for which tool. And the when isn’t obvious just by looking at Recall@K or Precision@K.

Because those metrics stop at the retrieval step. They tell you: “Yes, the system found relevant documents.” They don’t tell you:
– *Did the model actually use those documents?*
– *Did it fabricate details outside of context?*
– *Did the final answer address the user’s question fully?*
– *Was the answer grounded and faithful, or just eloquent hallucination?*

Let’s imagine: your retriever pulls three beautiful passages about Ashwagandha coffee—its adaptogenic properties, its calming effect, and its earthy taste. On Recall@K, you’d score high. Precision@K looks great too. But then the LLM answers: “Ashwagandha coffee tastes fruity like berries.”
Retrieval metrics will clap, but the user is misled.

This gap is exactly why the community shifted toward RAGAS evaluation. Instead of only auditing the pipes before the model, RAGAS evaluates the end-to-end experience: the question, the retrieved context, and the generated answer together.

It asks questions in the language of users:
– **Faithfulness:** did the answer stick to the documents, or did it invent?
– **Answer relevancy:** did the answer actually respond to the user’s query?
– **Context relevancy:** were the chunks retrieved meaningful for this query, or just vaguely on topic?
– **Context utilization:** did the model actually use the context it was given, or did it ignore most of it?

This is the bridge: everything you’ve learned so far about **Recall@K, Precision@K, rerankers, adaptive retrieval**—those are the knobs. RAGAS is the dashboard. It tells you when the knobs you’re turning are working, and when they’re not.

And here’s the hook: imagine running two pipelines side by side. One uses plain dense retrieval, the other uses multi-query + reranking. Traditional IR metrics might not reveal much difference once Recall@K hits a plateau. But run them through RAGAS, and suddenly you see: pipeline A produces answers with higher faithfulness but lower answer relevancy, while pipeline B produces more complete answers but sometimes hallucinates. Now you can take a call: which trade-off matters more for your product, your SLOs, your users.

That's why we need to move beyond retrieval metrics. Because a RAG system is not just a retriever—it's a conversation engine. And conversations are judged by trust, usefulness, and clarity, not just by whether the right PDFs were fetched behind the scenes.

Ragas Metrics

a. Offline (during testing / evaluation)

Here we *do* have ground truth (reference answers, annotated datasets, or distilled from docs). That allows **full evaluation**:

- **Context Recall**
  Did the retrieved chunks cover all the information in the reference answer?
  → Needs ground truth. Tells you if retriever is missing critical facts.
- **Context Precision**
  Of the retrieved chunks, how many are truly relevant to the query?
  → Needs ground truth. Detects noise in retrieval.
- **Answer Correctness / Semantic Similarity**
  How close is the model's answer to the reference answer?
  → Needs ground truth. Measures whether the final answer is "right."
- **Answer Coverage / Conciseness (extended)**
  Does the answer include all key aspects? Is it verbose or tight?
  → Optional, but often scored when ground truth is detailed.

Offline mode is where you run **regression tests, A/B comparisons, and benchmarks**.

b. Online (realtime guardrails / monitoring)

Here we *don't* have ground truth — only the query, retrieved context, and model's answer. So we focus on **consistency and safety checks**:

- **Faithfulness**
  Does the answer stay true to the retrieved context?
  → If low → hallucination risk.

- **Answer Relevancy**
  Does the answer actually respond to the user's query?
  → Prevents "fluent but off-topic" outputs.

- **Context Relevancy**
  Are the retrieved docs themselves on-topic for the query?

→ Quick retriever quality check without gold labels.

- **Context Utilization**
  Did the model actually use the retrieved context in its answer, or ignore it?
  → Detects wasted retrieval.

These are used as **guardrails**:

- If Faithfulness < threshold → regenerate with more docs or fallback to "I don't know."
- If Answer Relevancy < threshold → rephrase query, re-retrieve.
- Often computed with a smaller/faster LLM (Gemini Flash) to keep latency under control.

1. Base RAG pipeline

We first need a working RAG loop: documents → retriever → context → prompt → LLM → answer.

This is what we'll evaluate and later improve using RAGAS.

1.1 LLM Setup

We use Gemini as the reasoning engine. Notice we fix the temperature low (`0.2`) so the model doesn't wander creatively. This makes evaluation more stable:

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],
    temperature=0.2,
)
```

1.2 Context Formatting Helper

RAG answers depend heavily on how context is presented.
We'll format retrieved docs into a clean block with numbered `[S1], [S2]` tags — so that citations are consistent and easy to check later.

```
def format_context(docs):
    lines = []
    for i, d in enumerate(docs, 1):
        src = d.metadata.get("source", d.metadata.get("file_name",
f"S{i}"))
        snippet = (d.page_content or "").strip().replace("\n", " ")
        lines.append(f"[S{i}] {src}\n{snippet}")
    return "\n\n".join(lines)
```

1.3 Adding Conversation History

History keeps the conversation natural, but in RAG evaluation we treat it carefully — it should not introduce extra facts.

from langchain.schema import HumanMessage, AIMessage

```
history = [
    HumanMessage(content="I like spiced lattes but want healthier
options."),
    AIMessage(content="We can explore adaptogenic coffee variants."),
]
```

1.4 Prompt Template

We build a strict prompt with three parts: system instruction, conversation history, and retrieved context.
The instruction explicitly forces the model to **answer only from context** — this makes Faithfulness measurable.

```
from langchain_core.prompts import ChatPromptTemplate,
MessagesPlaceholder
```

```
prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a coffee assistant. "
     "Answer ONLY using the provided CONTEXT. If the answer is not in
CONTEXT, say so. "
     "Cite sources like [S1], [S2]. Be concise and actionable."
    ),
    MessagesPlaceholder(variable_name="history"),
    ("system", "CONTEXT:\n{context}"),
    ("human", "{question}")
])
```

1.5 Define the Retriever

```
# Define the retriever
from langchain_community.retrievers import
PineconeHybridSearchRetriever
retriever = PineconeHybridSearchRetriever(
    embeddings=embeddings,
    sparse_encoder=bm25,
    index=index,                # <-- pass the Index object
    top_k=8,
    alpha=0.5,                  # 0=dense, 1=sparse
    # namespace="docs",         # optional if you're using one
)
```

1.6 The RAG Chain

Now we connect prompt → LLM → parser.
This creates a runnable chain we can invoke repeatedly.

```
from langchain_core.output_parsers import StrOutputParser
chain = prompt | llm | StrOutputParser()
```

1.7 Retrieval + Execution

For now we assume you already have a retriever (dense, hybrid, etc.).
We'll run a sample query, format context, and generate the answer.

```
# Example query
query = "What is Ashwagandha coffee and how does it taste?"

# Retrieve supporting documents
ctx_docs = retriever.get_relevant_documents(query)
ctx_str  = format_context(ctx_docs)

# Invoke chain
answer = chain.invoke({
    "history": history,
    "context": ctx_str,
    "question": query,
})

print(answer)
```

1.8 Why This Base Matters

This base pipeline is our *first draft*. It runs end-to-end: retrieval, context packaging, prompt, LLM, and answer. But in practice, no first draft survives contact with evaluation. RAGAS will shine a light on where this system falls short — and each section above is a potential tuning knob:

- **LLM setup (1.1)**
  If RAGAS shows low *faithfulness* (the model hallucinates outside context), we may tighten temperature, or even swap to a model variant that prioritizes grounding.

- **Context formatting (1.2)**
  If *context utilization* is low, maybe the formatting is noisy — too much boilerplate, not enough concise snippets. We may need to compress chunks or standardize citation tags so the model can use them reliably.

- **History injection (1.3)**
  If answers start drifting, mixing old history with new facts, RAGAS will flag poor *faithfulness*. We'd revisit how we separate memory (conversational flow) from factual

context (retrieved docs).

- **Prompt template (1.4)**
  If evaluation shows low *answer relevancy* (model skirts around the actual question), or citations are missing, we'll adjust the system instruction — maybe enforce structured JSON or stricter citation rules.

- **RAG chain construction (1.5)**
  If answers are too verbose or inconsistent, we might insert a post-answer verification step (self-check, corrective generation) before parsing.

- **Retrieval (1.6)**
  If *context precision* or *context recall* is weak, we'll revisit the retriever: add reranking, enable hybrid retrieval, or filter by metadata.

The key insight: **RAGAS doesn't just score us — it points us back to which layer in the pipeline needs fixing.** Retrieval metrics (Recall@K, Precision@K) alone only say "retriever good/bad." RAGAS tells us whether our prompt, formatting, or even LLM behavior is sabotaging the final answer.

This is why we start with a clean baseline. Every subsequent improvement will be an iteration loop: *run pipeline → evaluate with RAGAS → decide which section to revisit → adjust → re-evaluate*.

Offline Metrics

1. Context Recall

1.1 Why start here?

You and your students already know retrieval metrics like Recall@K and Precision@K. RAGAS starts from the same intuition but aligns it with *what the user asked*.

- Recall here = *"did the retrieved docs contain enough information to actually answer the question?"*
- This is not about whether the docs are semantically close, but whether they truly carry the ground-truth answer.

So Recall bridges what they know (retriever quality) with where we're going (end-to-end RAG quality).

$$\text{Context Recall} = \frac{\#\text{ of relevant retrieved chunks}}{\text{Total relevant chunks in corpus}}$$

Step 1. Define a test case

We'll reuse your query:

```
query = "What is Ashwagandha coffee and how does it taste?"
answer = chain.invoke({
    "history": history,
    "context": format_context(ctx_docs),
    "question": query,
})
print(answer)
```

Now suppose the **ground-truth answer** we want is something like:

"Ashwagandha coffee is a herbal coffee infused with ashwagandha root, often described as earthy and slightly bitter, with adaptogenic health benefits."

Step 2. Measure Context Recall

Install RAGAS first:

```
pip install ragas
```

```
from ragas.metrics import context_recall
from ragas.evaluators import evaluate
from ragas.dataset import Dataset

# Build dataset for evaluation
data = Dataset.from_dict({
    "question": [query],
    "answer": [answer],
    "contexts": [[d.page_content for d in ctx_docs]],  # retrieved
docs
```

```
    "ground_truth": [
        "Ashwagandha coffee is infused with ashwagandha root and
tastes earthy, slightly bitter, with calming effects."
    ],
})

# Evaluate Recall
results = evaluate(data, metrics=[context_recall])
print(results)
```

This will give you a score between `0–1`.

- `1.0` = the retrieved chunks definitely contain all the facts needed.
- `0.0` = nothing useful was retrieved.

Step 3. How to improve based on Context Recall

- **If recall is low**: it means the retriever didn't bring in the necessary info.
  → Try increasing `top_k` from 8 to 15.
  → Try lowering `alpha` towards `0.3` to lean more on dense semantic retrieval.
  → Or apply **multi-query expansion** to cover variations of "Ashwagandha coffee taste".

- **If recall is already high**: you don't need to adjust retrieval, but the next weak link may be re-ranking or context prep.

Notice what we're doing:

- Retrieval metrics you already know (Recall@K) → **Context Recall in RAGAS** is just the RAG-specific framing of the same idea.
- But now, we're not measuring similarity, we're asking *did we pull in the right knowledge to answer the question truthfully?*

1.2 Ways to Get Ground Truth in RAGAS

a. Manual Gold Standard (Human Annotated)

- You (or annotators) write the *ideal reference answers* for a set of questions.

- Example: For `"What is Ashwagandha coffee and how does it taste?"`, the reference might be:
  "Ashwagandha coffee is made by infusing roasted coffee with powdered ashwagandha root. It tastes earthy, slightly bitter, and is believed to have calming properties."
- **Pros:** Very precise, high-quality benchmarks.
- **Cons:** Expensive and slow to scale.

This is often how academic QA datasets like *HotpotQA* or *NaturalQuestions* are built.

b. Use Your Document Corpus

- Instead of writing answers yourself, you extract the **source passages** directly from your knowledge base.
- For our coffee docs: if one chunk explicitly says *"Ashwagandha coffee tastes earthy and bitter"*, that passage can serve as the ground truth.
- **Pros:** Scales automatically, low effort.
- **Cons:** These are fragments, not full answers — you'll need to normalize them (maybe with summarization).

This is especially common when you already trust your docs as the "source of truth."

c. LLM-as-Judge (Reference Generation)

- You ask a *trusted LLM* to generate a reference answer from your document corpus.
- Example:

```
"Given these documents, generate the most factual, concise answer to the question."
```

- That generated answer becomes the ground truth for evaluation.
- **Pros:** Quick, automates dataset creation.
- **Cons:** Risk of model hallucination if it strays outside the provided context.

d. Existing Benchmark Datasets

- Plug into public datasets where questions already have labeled answers (e.g., *TriviaQA*, *SQuAD*, *HotpotQA*).
- Then run your pipeline on those same questions and compare.
- **Pros:** Standardized, comparable to other systems.
- **Cons:** May not match your domain (e.g., coffee domain doesn't exist in benchmarks).

e. Weak Ground Truth via Majority Voting

- Generate multiple candidate answers (from models, docs, or past runs).
- Use majority vote or semantic clustering to decide on a "pseudo ground truth."
- **Pros:** No single annotator bias, useful in scaling.
- **Cons:** Still weaker than a true curated gold set.

How RAGAS Uses Ground Truth

- For **Context Recall**: checks if retrieved chunks *contain* the ground truth facts.
- For **Answer Faithfulness**: compares generated answer against the context, not against the reference.
- For **Answer Relevance / Correctness**: compares generated answer with the ground truth reference.

So depending on the metric, you might not need full human-written answers — sometimes just **the relevant supporting chunks** are enough.

Code to calculate Context Recall

```
import os
os.environ["GIT_PYTHON_REFRESH"] = "quiet"  # silence git check

from ragas.evaluation import evaluate
from ragas.metrics import context_recall
from ragas.dataset_schema import SingleTurnSample, EvaluationDataset

# Import Gemini wrapper from LangChain
from langchain_google_genai import ChatGoogleGenerativeAI

# Wrap Gemini into LangChain LLM
gemini_llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],
    temperature=0.0,
)

# Build a single test sample
sample = SingleTurnSample(
    user_input="What is Ashwagandha coffee and how does it taste?",
```

```
    response=answer,
    retrieved_contexts=[d.page_content for d in ctx_docs],
    reference="Ashwagandha coffee is infused with ashwagandha root "
              "and tastes earthy, slightly bitter, with
calming/adaptogenic effects."
)

dataset = EvaluationDataset(samples=[sample])

# Run evaluation using Gemini instead of OpenAI
results = evaluate(dataset=dataset, metrics=[context_recall],
llm=gemini_llm)

print(results)
```

2. Context Precision

2.1 Why it matters

Recall told us: *"Did we bring in enough facts to answer the question?"*
But high recall alone isn't enough — a retriever can spam you with 20 docs, hoping the answer is buried somewhere. That bloats context, slows down the LLM, and risks hallucinations.

Context Precision flips the lens:
"Of the documents I pulled, how many were actually relevant to the user's question?"

- High Precision → concise, on-topic context (LLM has less noise).
- Low Precision → lots of junk in context (LLM may get distracted).

So recall = coverage, precision = focus.

2.2 How RAGAS measures it

- Input: `(query, retrieved_contexts, ground truth reference)`
- It checks, for each retrieved chunk, *does this chunk really contribute to answering the query / align with the reference?*

- Then it computes the ratio:

$$\text{Context Precision} = \frac{\#\text{ of relevant retrieved chunks}}{\text{Total retrieved chunks}}$$

2.3 Code Example (with Gemini as judge)

```
import os
os.environ["GIT_PYTHON_REFRESH"] = "quiet"

from ragas.evaluation import evaluate
from ragas.metrics import context_precision
from ragas.dataset_schema import SingleTurnSample, EvaluationDataset
from langchain_google_genai import ChatGoogleGenerativeAI
```

```
# Wrap Gemini
gemini_llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],
    temperature=0.0,
)

# Build test sample
sample = SingleTurnSample(
    user_input="What is Ashwagandha coffee and how does it taste?",
    response=answer,                                    # from your
chain
    retrieved_contexts=[d.page_content for d in ctx_docs],
    reference="Ashwagandha coffee is infused with ashwagandha root "
              "and tastes earthy, slightly bitter, with
calming/adaptogenic effects."
)

dataset = EvaluationDataset(samples=[sample])

# Evaluate Precision
```

```
results = evaluate(dataset=dataset, metrics=[context_precision],
llm=gemini_llm)

print(results)
```

2.4 How to improve based on Precision

- **If precision is low** (retriever fetched too much noise):
  - Reduce `top_k` (maybe from 15 → 8).
  - Add **reranker** (cross-encoder or embedding similarity) to prioritize tighter matches.
  - Apply **metadata filters** (e.g., domain, timestamp).
  - Compress or clean context before passing to LLM.
- **If precision is already high but recall is low** → you're too strict; expand top_k or allow hybrid queries.

3. Answer Correctness / Semantic Similarity

3.1 Why it matters

Even if:

- **Recall** is high (we pulled the right facts), and
- **Precision** is high (we didn't pull junk),

…the model might still produce a **bad answer**. Why?

- It might ignore some facts.
- It might be rephrased incorrectly.
- It might add hallucinated fluff.

That's why we need a metric that directly checks:
"How close is the model's answer to the ground-truth answer we expect?"

This is **Answer Correctness** (sometimes called Semantic Similarity).

3.2 How RAGAS measures it

- Input: `(query, generated answer, ground-truth reference)`

- Uses an embedding-based semantic similarity (or an LLM judge in stricter mode) to compare the model's answer with the ground truth.
- Score:
  - **1.0** → answer semantically matches the reference fully.
  - **0.0** → completely off or contradictory.
  - Values in between reflect partial coverage.

It's not string overlap (like BLEU or ROUGE), but **semantic equivalence**.

3.3 Code Example (Gemini as judge)

```
import os
os.environ["GIT_PYTHON_REFRESH"] = "quiet"

from ragas.evaluation import evaluate
from ragas.metrics import answer_relevancy   # this is the metric
RAGAS uses for correctness/similarity
from ragas.dataset_schema import SingleTurnSample, EvaluationDataset
from langchain_google_genai import ChatGoogleGenerativeAI

# Wrap Gemini
gemini_llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],
    temperature=0.0,
)

# Ground-truth answer for testing
reference_answer = (
    "Ashwagandha coffee is infused with ashwagandha root. "
    "It tastes earthy and slightly bitter, and is often associated
with calming/adaptogenic effects."
)

# Build sample
sample = SingleTurnSample(
    user_input="What is Ashwagandha coffee and how does it taste?",
    response=answer,                                    # your
model's answer
    retrieved_contexts=[d.page_content for d in ctx_docs],
```

```
    reference=reference_answer
)
dataset = EvaluationDataset(samples=[sample])
# Evaluate Answer Correctness / Similarity
results = evaluate(dataset=dataset, metrics=[answer_relevancy],
llm=gemini_llm)

print(results)
```

3.4 How to improve based on Correctness

- **If correctness is low but recall/precision are high** → the **LLM is at fault**.
    - Revisit the **prompt**: enforce stricter grounding ("Answer ONLY from context").
    - Lower temperature to reduce creative drift.
    - Add a **self-check step**: have the LLM verify whether each sentence is supported by context.
- **If correctness and recall are both low** → the retriever is failing, not the generator.
- **If correctness is low but faithfulness is high** → the answer is true to context, but incomplete. Maybe retrieved docs don't cover all aspects of the reference.

4. Other metrics

- **Answer Semantic Similarity** → How close is the generated answer to the gold answer semantically?
- **Context Entities Recall** → Did the retrieved docs include all the key entities present in the reference answer?
- **Aspect Critique** → Evaluate answer against specific aspects (e.g., coverage, style, tone) defined in reference guidelines.
- **Domain-Specific Evaluation** → Needs domain-specific gold standards (finance, healthcare, etc.).
- **Summarization Score** → If summarization is the task, compare generated vs reference summaries.
  https://docs.ragas.io/en/v0.1.21/concepts/metrics

Online Metrics

So far, we looked at **offline metrics** (Context Recall, Precision, Correctness).

- They are powerful for **testing** because we have a *ground-truth reference*.
- But in **production/live use**, there is no reference answer — otherwise you wouldn't need RAG at all.

That's where **online metrics** come in. They don't rely on a gold answer; instead, they use the *retrieved context* itself to sanity-check the model's response.

1. Faithfulness

1.1 Why start here?

Even if the retriever did its job, the model might still:

- invent facts,
- overstate what's in the docs, or
- stitch together something that sounds fluent but isn't supported.

So the key question becomes:
"Is the answer faithful to the retrieved context, or did the model hallucinate?"

That's why faithfulness is often the **#1 online guardrail metric**.

1.2 How it works (behind the scenes)

1. Break the generated answer into factual statements.
2. For each statement, check if it is *entailed by* at least one of the retrieved docs.
    - This is done by the evaluation LLM (Gemini, Claude, etc.) acting as a judge.
3. Score = % of answer statements that are supported by the context.

So faithfulness is like:

- *"Line by line, did you back up what you said with the books you looked at?"*

1.3 Example with our pipeline

Query: *"What is Ashwagandha coffee and how does it taste?"*

- Retrieved context mentions: earthy, slightly bitter, calming effects.

- Model answer: *"Ashwagandha coffee is a sweet chocolate-flavored drink from South America."*

→ Recall could still be high (retrieved the right docs).
→ But faithfulness will be **0.0**, because the answer is **not supported** by the retrieved context.

```
import os
os.environ["GIT_PYTHON_REFRESH"] = "quiet"

from ragas.evaluation import evaluate
from ragas.metrics import faithfulness
from ragas.dataset_schema import SingleTurnSample, EvaluationDataset
from langchain_google_genai import ChatGoogleGenerativeAI

# Use Gemini as evaluation judge
gemini_llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],
    temperature=0.0,
)

sample = SingleTurnSample(
    user_input="What is Ashwagandha coffee and how does it taste?",
    response=answer,                                    # model's
output
    retrieved_contexts=[d.page_content for d in ctx_docs],
    reference=None  # not needed for faithfulness
)

dataset = EvaluationDataset(samples=[sample])

results = evaluate(dataset=dataset, metrics=[faithfulness],
llm=gemini_llm)
print(results)
```

1.4 How to use it in practice (online)

- **Guardrail at serving time:**
  - If faithfulness score ≥ threshold (e.g. 0.7) → return answer.

    - If < threshold → regenerate with larger top_k, or fallback ("I don't know from context").
- **Monitoring:**
  Log faithfulness scores on live traffic to spot hallucination spikes (e.g., after retriever changes).

1.5 Using Faithfulness in a Live Improvement Loop

In testing, you only *observe* scores.
But in production, you can **act on faithfulness scores in real-time** to keep the system honest.

Here are some practical strategies:

a. Threshold-based Guardrail

- Set a minimum faithfulness score (say **0.7**).
- At runtime:
  - If score ≥ 0.7 → return the answer to the user.
  - If score < 0.7 → trigger fallback.
    - e.g., re-retrieve with higher `top_k`
    - or return: *"I don't have enough information in the provided documents."*

This is the simplest loop: score → filter → fallback.

b. Regeneration with Expanded Context

If faithfulness is low:

1. Expand the search: increase `top_k` (from 8 → 15).
2. Rebuild the context.
3. Regenerate the answer.
4. Re-check faithfulness before responding.

This loop is often called **"retry with more evidence."**

c. Self-Corrective Prompting

Use the LLM itself to repair an unfaithful answer:

- Input: *"Here's your previous answer and the retrieved context. Rewrite the answer so it is fully supported by the context only."*

- This way, the LLM becomes both generator and corrector.

d. Logging + Active Learning

- Log low-faithfulness cases into a database.
- Review them offline to:
  - improve retriever (add better metadata, chunking, hybrid search tuning),
  - or refine prompts.
- Over time, this creates a feedback loop: production signals improve the system continuously.

e. Selective Escalation

- For high-risk domains (legal, finance, healthcare):
  - Route low-faithfulness answers to human reviewers instead of showing them live.
- This blends automation with human oversight safely.

2. Answer Relevancy

2.1 Why it matters

Faithfulness asked:
"Did the answer stay true to the retrieved context?"

But even if the answer is faithful, it might still be **irrelevant** to what the user asked.

- Example:
  - **Question:** *"What is Ashwagandha coffee and how does it taste?"*
  - **Retrieved context:** Talks about health benefits, origins, and taste.
  - **Model's answer:** *"Ashwagandha coffee is used in Ayurvedic medicine for stress relief."*
  - → This is *faithful* (the docs mention it).
  - → But *not relevant* (the user asked about taste, not health effects).

So, **Answer Relevancy = Did the answer actually address the user's question?**

2.2 How RAGAS measures it

- Takes `(user_input, response)` (and sometimes `reference` if available).

- An evaluation LLM (Gemini, Claude, etc.) is asked:
  "Does the response directly answer the question?"
- Score:
  - **1.0** → fully relevant.
  - **0.0** → totally irrelevant or off-topic.
  - Intermediate values for partial relevance.

It's not about factuality (that's faithfulness), it's about *focus*.

2.3 Example with our pipeline

```
import os
os.environ["GIT_PYTHON_REFRESH"] = "quiet"

from ragas.evaluation import evaluate
from ragas.metrics import answer_relevancy
from ragas.dataset_schema import SingleTurnSample, EvaluationDataset
from langchain_google_genai import ChatGoogleGenerativeAI

# Use Gemini as evaluation judge
gemini_llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],
    temperature=0.0,
)

sample = SingleTurnSample(
    user_input="What is Ashwagandha coffee and how does it taste?",
    response=answer,                                    # model's
answer
    retrieved_contexts=[d.page_content for d in ctx_docs],
    reference=None   # not needed for relevancy
)

dataset = EvaluationDataset(samples=[sample])

results = evaluate(dataset=dataset, metrics=[answer_relevancy],
llm=gemini_llm)
print(results)
```

2.4 How to use it in practice (online)

- **Guardrail at serving time:**
  - If relevancy ≥ threshold (say 0.8) → return answer.
  - If < threshold → re-check query formulation or fallback.
- **Query reformulation loop:**
  If answer is irrelevant → regenerate after rephrasing the user query (e.g., multi-query **expansion**).
- **Monitoring:**
  Track relevancy in logs. A drop often signals that the retriever drifted or the LLM ignored the query focus.

2.5 Improvement strategies

- If relevancy is low but faithfulness is high →
  → The answer is true but not aligned with the query. Fix with **better prompt instructions**:
  "Always answer the user's exact question. Do not provide extra details unless asked."
- If relevancy is low and faithfulness is also low →
  → Likely retriever failure. Improve retrieval quality first.

3. Context Utilization

3.1 Why it matters

Recall/Precision tell you what you *pulled*. Faithfulness tells you if the answer stayed true to that context. **Context Utilization** asks a different, very practical question:
"Of what you retrieved, how much did the model actually USE?"
Low utilization = you're over-fetching (higher cost/latency, more noise). High utilization = tight, efficient contexts the model really leveraged.

3.2 What RAGAS measures

At a high level, RAGAS scores the fraction of retrieved chunks that *contributed support* to the answer (or, depending on version, the proportion of answer content that's actually grounded in the retrieved chunks). It uses an LLM judge (and sometimes embeddings) to mark which chunks support which claims, then computes a 0–1 score.

- **1.0** → the answer leaned on most of what you fetched (or most of the answer is grounded).

- **0.0** → the answer ignored the retrieval or used none of it.

3.3 Code

```
import os
os.environ["GIT_PYTHON_REFRESH"] = "quiet"

from ragas.evaluation import evaluate
from ragas.metrics import context_utilization
from ragas.dataset_schema import SingleTurnSample, EvaluationDataset
from langchain_google_genai import ChatGoogleGenerativeAI

# Use Gemini as evaluation judge
gemini_llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],
    temperature=0.0,
)

sample = SingleTurnSample(
    user_input="What is Ashwagandha coffee and how does it taste?",
    response=answer,                                    # model's
answer
    retrieved_contexts=[d.page_content for d in ctx_docs],
    reference=None   # not needed for relevancy
)

dataset = EvaluationDataset(samples=[sample])

results = evaluate(dataset=dataset, metrics=[context_utilization],
llm=gemini_llm, embeddings = embeddings)
print(results)
```

3.4 How to act on it (live loop)

- **Low utilization with decent recall/faithfulness** → you're over-fetching or fetching redundant chunks.
    - Reduce `top_k` (e.g., 15 → 8).
    - Switch your retriever to **MMR** to enforce diversity.
    - Add a **cross-encoder reranker** before prompting (keep top 6–10).
    - Wrap with **ContextualCompressionRetriever** (LLM extractor) to trim long/noisy chunks.
    - De-duplicate near-identical chunks (e.g., `EmbeddingsRedundantFilter`).
- **Low utilization + low faithfulness** → often retrieval mismatch; revisit query-rewrite (multi-query, self-query filters), adjust `alpha` in hybrid, or improve metadata/namespace routing.
- **High utilization but low correctness** → LLM synthesis problem; tighten prompt ("Answer ONLY from context"), lower temperature, add **self-check + repair** pass.

What to check when?

Table 1:

Case	What You See	Metric Signals	Likely Cause	Where to Fix	Knobs / Concrete Moves	Quick Tests (fast)	Telemetry to Log
Low Recall	Missing key facts	Low Recall@K, low RAGAS Context Recall	Retriever not covering; k too small; bad embedding space; overly tight filters	Retrieve	↑top_k to 24–40; enable multi-query; hybrid α=0.30–0.45; widen recency filter; add fetch_k=60 + MMR (λ≈0.3–0.5)	Ask same Q with: (a) query expansion; (b) filters off. Compare Recall@K.	Chosen k, alpha, filter JSON, #candidates before/after MMR, %dropped by filters
High Recall, Low Precision	Lots of irrelevant chunks	High Context Recall, low Context Precision	Over-fetching, fuzzy filters, no rerank	Prepare	Add cross-encoder rerank (pool 40→keep 8–12); tighten filters; reduce top_k by 20–40%; raise MMR λ (more diversity); cap max per-doc = 2–3	A/B with CE on/off; track top-12 precision difference	CE score distro, kept vs dropped counts, per-doc cap hits

High Recall & Precision, Low Correctness	Facts present, answer wrong	RAGAS Answer Correctness low; Faithfulness OK	Prompt lets model reason beyond context; temp too high; wrong synthesis template	Prompt / Generate	Lower temperature ≤ 0.2; switch to compact-and-refine or "lookup" style; add "answer only from CONTEXT" + step-wise citation rules	Freeze context, swap prompt only; compare correctness delta	Prompt version hash, temp, tokens, #claims with citations
Low Faithfulness	Hallucinations / uncited claims	Low Faithfulness; extra claims	Generator fabricates; prompt vague; missing refusal line	Prompt / Guardrails	Add refusal line + JSON schema for claims+citations; 2-pass repair ("rewrite strictly from CONTEXT"); block "no-source" sentences	Turn on repair once; expect ≥+0.1 faithfulness	Repair triggered? yes/no; judge scores pre/post; fallback used?
Low Answer Relevancy	Off-topic answer	Low Answer Relevancy; high Context Precision	Misread query; history leakage; prompt intent wrong	Prompt / Retrieve	Add query reformulation step; use branch prompts (compare/explain/lookup); trim history via MessagesPlaceholder windowing	Run once with history cleared; once with decomposition	Reformulated query, branch chosen, history window size
Low Context Utilization	Model ignores context	Low Context Utilization; high token context	Redundant chunks; over-compression; poor ordering	Prepare	Long-context reorder (lead with most specific); dedup near-dupes; reduce top_k 20–40%; compression lighter for numeric	Compare utilization after reordering only	Position of cited chunks, utilization %, dedup count
Wrong / Missing Citations	Good answer, bad cites	Faithfulness borderline; citation mismatch	Passage IDs lost; summarizer dropped numbers; chunk boundaries off	Prepare / Prompt	Preserve [DocID#Chunk]; disable aggressive compression on numeric tables; SentenceWindow nodes	Switch to sentence-window parser; check if numbers survive	%answers with valid doc/offset; numeric-token retention
Over-Compression	Numbers/units clipped	Correctness falls only on numeric Qs	Summarizer too aggressive	Prepare	Compression off for "numeric" class; re-rank → keep 8–10 raw chunks; expand context budget	Classify queries; compare numeric subset	Class decision, compressor on/off flag, token budget used
Filters Silence Results	"I don't know" too often	Very low candidate count	Metadata filter too tight; wrong type	Retrieve	Log candidate count pre-filter; relax one filter at a time; add guard: if 0 hits → retry without filters	Simulate with filters removed; expect ≥1 hit	Pre/post filter counts; filter keys used

Embedding Drift	Recall regresses after re-index	Drop vs previous build	Mixed models/dims; normalization changed	Ingest / Indexing	Pin embedding model version; rebuild entire index with same model; store embed_model_id in metadata	AB compare old/new on a held-out set	Index version, embed model id, mean similarity shift
Namespace Leakage	Cross-corpus answers	Irrelevant sources cited	Wrong namespace or filter	Retrieve	Ensure namespace per corpus; enforce namespace in retriever; add doc_type filter	Ask a corpus-specific query; ensure only that corpus appears	Namespace name, %out-of-corpus hits
Freshness Drift	Old policies answered	Wrong date in answer	Stale index; missing recency filter	Ingest / Retrieve	Nightly mini-ingest; add PARTITION_DATE filter window; boost recency in hybrid fusion	Ask dated Q; check newest doc date in context	Doc timestamps, recency boosts, index refresh time
Latency Spike	p95 > budget	SLO breach	k too high; CE + compression + retry	System knobs	Branching: cheap path for narrow Qs; cap CE pool 20–30; disable compression on fast path	Flag “narrow” queries via heuristics; compare p95	Path taken (fast/slow), stages timing, p95/avg
Cost Spike	Token /$ jumps	Alerts	Over-large contexts; repeated retries; cache miss	System knobs	Add retrieval cache on normalized queries; answer cache gated by doc hash; token cap per stage	Same query twice → expect 1 cache hit	Cache hit rate, token per stage, retry count

4. Making an enhancement loop with RAGAS

4.1 Build a tiny eval sample from your current run

```
# Build an eval sample from the live run you just executed
from ragas.dataset_schema import SingleTurnSample, EvaluationDataset

sample = SingleTurnSample(
    user_input=q,
    response=answer,
    retrieved_contexts=[d.page_content for d in ctx_docs],
    # For recall, give a light reference line (you can hand-write)
    reference="Ashwagandha coffee is infused with ashwagandha root
and tastes earthy, slightly bitter."
```

```
)
dataset = EvaluationDataset(samples=[sample])
```

4.2 Evaluate with RAGAS

```
import os
os.environ["GIT_PYTHON_REFRESH"] = "quiet"

from ragas.evaluation import evaluate
from langchain_google_genai import ChatGoogleGenerativeAI,
GoogleGenerativeAIEmbeddings

# Judge LLM: Gemini
gemini_llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key=os.environ["GEMINI_API_KEY"],
    temperature=0.0,
)

# Embeddings backend: Gemini (prevents OpenAI default)
gemini_emb = GoogleGenerativeAIEmbeddings(
    model="models/embedding-001",  # or text-embedding-004 if enabled
in your account
    google_api_key=os.environ["GEMINI_API_KEY"],
)

# Pick metrics (safe trio: recall = offline-ish;
faithfulness/relevancy = online)
from ragas.metrics import context_recall, faithfulness,
answer_relevancy

results = evaluate(
    dataset=dataset,
    metrics=[context_recall, faithfulness, answer_relevancy],
    llm=gemini_llm,
    embeddings=gemini_emb
)
```

```
print(results)
```

4.3 Auto-improve loop (simple knobs: `top_k` & `alpha`) using RAGAS scores

```
def ragas_score(q, answer_text, docs):
    # Rebuild a one-sample dataset
    s = SingleTurnSample(
        user_input=q,
        response=answer_text,
        retrieved_contexts=[d.page_content for d in docs],
        reference="Ashwagandha coffee is infused with ashwagandha
root and tastes earthy, slightly bitter."
    )
    ds = EvaluationDataset(samples=[s])
    r = evaluate(
        dataset=ds,
        metrics=[context_recall, faithfulness, answer_relevancy],
        llm=gemini_llm,
        embeddings=gemini_emb
    )
    # pull scalar numbers
    return {
        "context_recall": float(r["context_recall"]),
        "faithfulness": float(r["faithfulness"]),
        "answer_relevancy": float(r["answer_relevancy"]),
    }

def run_once(q, top_k=None, alpha=None):
    # set knobs if supported by your retriever
    if top_k is not None and hasattr(retriever, "top_k"):
retriever.top_k = top_k
    if alpha  is not None and hasattr(retriever, "alpha"):
retriever.alpha  = alpha

    docs = retriever.get_relevant_documents(q)
    ctx  = format_context(docs)
```

```
    ans  = chain.invoke({"history": history, "context": ctx,
"question": q})
    scores = ragas_score(q, ans, docs)
    return {"answer": ans, "docs": docs, "scores": scores, "top_k":
top_k or getattr(retriever,"top_k",None), "alpha": alpha or
getattr(retriever,"alpha",None)}

# Try a small grid of improvements
candidates = []
candidates.append(run_once(q))                          # current
candidates.append(run_once(q, top_k=15))                # ↑ recall
candidates.append(run_once(q, top_k=15, alpha=0.3))     # ↑ recall +
lean dense
candidates.append(run_once(q, top_k=10, alpha=0.4))     # moderate
tweak

# Pick best: prioritize Faithfulness, then Relevancy, then Recall
def rank_key(c):
    s = c["scores"]
    return (round(s["faithfulness"], 4), round(s["answer_relevancy"],
4), round(s["context_recall"], 4))

best = max(candidates, key=rank_key)

print("Chosen knobs: top_k=", best["top_k"], "alpha=", best["alpha"])
print("Scores:", best["scores"])
print("Answer:\n", best["answer"])
```

RAG System Design

Below is the end-to-end skeleton — **ingest → retrieve → prepare → prompt → generate** — mapped to the abstractions you've seen in **LangChain** and **LlamaIndex**, along with the knobs you've been tuning and the common pitfalls you've already run into.

1. RAG Skeleton Recap

1.1 Ingest (load & parse raw sources)

Goal: turn raw HTML/PDF/MD/Docs/URLs/APIs into clean `Document`s with useful **metadata**.

- **What you did:** HTML with headings, stripped boilerplate, added `title/heading_keywords/ts/source_url`.
- **LangChain:** `DirectoryLoader`, `Unstructured*Loader`, `BSHTMLLoader`; custom cleaners; attach metadata on load.
- **LlamaIndex:** `SimpleDirectoryReader`, `NodeParser` for structure; `MetadataExtractor`.
- **Hot knobs:** parser choice, HTML → text policy, metadata richness (section path, page #, timestamps).
- **Pitfalls:** over-cleaning (losing tables/units), missing canonical `doc_id`, inconsistent metadata types.

1.2 Retrieve (candidate search over the index)

Goal: given a query, fetch **relevant candidates** (dense, sparse, or hybrid) with optional filters.

- **What you did:** Pinecone index; **hybrid** via BM25 + embeddings; namespace per corpus; filters like `heading_keywords`, recency windows.
- **LangChain:** `PineconeVectorStore.as_retriever`, `BM25Retriever`, **hybrid** (Ensemble or Hybrid retrievers), `SelfQueryRetriever`, `MultiQueryRetriever`.
- **LlamaIndex:** `VectorStoreIndex` + `RetrieverQueryEngine`, `BM25Retriever`, `QueryFusionRetriever`, `SelfQuery`/`MultiStepQueryEngine`.
- **Hot knobs:** `k`, `alpha` (dense vs sparse), metadata filters, namespaces, `fetch_k` for MMR pools.
- **Pitfalls:** too small `k` for broad asks, filters that silently drop everything, drifting embeddings across re-indexes.

1.3 Prepare (shortlist, clean, compress, budget)

Goal: turn many candidates into a **compact, diverse, de-duplicated** context block that fits your token budget.

- **What you did:** Cross-encoder re-rank; MMR for diversity; dedup near-identical chunks; compression/summarize; pack under a token cap; preserve `[DocID]`.
- **LangChain:** `CrossEncoder` rerankers, `mmr` retriever, `EmbeddingsRedundantFilter`, `ContextualCompressionRetriever`, `LongContextReorder`.
- **LlamaIndex:** `Reranker` (e.g., Cohere/CE), `SentenceWindowNodeParser`, `NodePostprocessors` (Similarity/LLM extract), `SummaryCompressor`, `ContextWindowManager`.
- **Hot knobs:** re-rank size (e.g., 40→10), MMR `lambda`, max per-doc passages, compression aggressiveness, total token budget.
- **Pitfalls:** over-compression that drops numbers/citations, letting one doc monopolize context, not normalizing scores before fusion.

1.4 Prompt (shape the contract)

Goal: separate user input, retrieved context, history/memory; impose citation & output rules.

- **What you did:** system message with "answer only from CONTEXT", explicit citation style `[S#]`, optional JSON mode for validators, clean history via `MessagesPlaceholder`.
- **LangChain:** `ChatPromptTemplate`, `MessagesPlaceholder`, `StrOutputParser`/`JsonOutputParser`, runnable pipes `prompt | llm | parser`.
- **LlamaIndex:** `PromptTemplate`/`ChatPromptTemplate`, `ResponseSynthesizers` (Refine/Tree/CompactAndRefine), `StructuredOutputParser`.
- **Hot knobs:** refusal line, schema (JSON vs text), history window, memory section vs doc context separation.
- **Pitfalls:** leaking history into facts, vague instructions on citations, prompts that encourage speculation.

1.5 Generate (LLM answering)

Goal: produce a concise, **grounded** answer with citations; optionally stream; optionally verify/repair.

- **What you did:** Gemini Flash for answers; optional self-check/repair pass; low temperature for grounded QA; streaming when interactive.

- **LangChain:** `ChatGoogleGenerativeAI` (or other chat models), streaming callbacks, post-validators (faithfulness/relevancy chains), retry on failure.
- **Hot knobs:** temperature, max tokens, model tier (flash vs pro), single- vs two-pass (verify/repair), thresholds for pass/fail.
- **Pitfalls:** too "creative" temps, skipping verification, not surfacing "I don't know" when context is thin.

2. From Components to Systems

You've now seen the **skeleton** of a RAG pipeline:
ingest → retrieve → prepare → prompt → generate.

You've also explored all the **lego blocks** inside (loaders, retrievers, rerankers, prompt templates, LLM callers).
But here's the reality:

> **Real-world RAG isn't about knowing the parts, it's about how you assemble them for your use case.**
> Two systems might use the exact same retriever and embeddings, yet one works reliably and the other fails. The difference is in how you **design the system** around those blocks.

This is the shift from *components → systems thinking*.

2.1 Skeleton vs System

- **Skeleton:** a static pipeline of steps. Same path every time, same knobs (e.g., always retrieve top_k=8, always pass context into the same prompt).

- **System:** a pipeline that adapts, validates, and sometimes retries. It knows when to say *"I don't know,"* when to expand search, and when to refuse unsafe questions.

Analogy for students: Skeleton = recipe, System = chef.
The recipe lists ingredients, but the chef adjusts salt, adds extra garnish, or re-cooks if it's underdone.

2.2 Decision Axes in System Design

When moving from skeleton → system, you face tradeoffs along key axes:

1. **Latency vs Accuracy vs Cost**

    - More `top_k`, reranking, and validators = higher accuracy, but slower and more expensive.
    - Example: Customer support chatbot with p95 latency budget = 1.5s → you can't afford a cross-encoder + 3 repair loops.
2. **Open-domain vs Narrow-domain**
    - Open domain (Wikipedia-like) → broad queries, need adaptive multi-query retrievers.
    - Narrow domain (internal policies) → high risk if you hallucinate → corrective loops and validators are mandatory.
3. **Ground-truth availability**
    - If you have gold answers (training/eval), you can run **offline metrics** (recall, correctness).
    - In production, you only have **online metrics** (faithfulness, relevancy).
4. **Failure tolerance**
    - Can you safely return "I don't know"? (FAQ chatbot = yes. Legal/medical = must refuse if unsure. Marketing copy = maybe add fallback fluff.)
5. **Freshness vs Stability**
    - Do you need new docs every day (live ingestion) or is weekly indexing enough?
    - Freshness demands adaptive RAG design (dynamic filters by timestamp). Stability lets you pre-tune offline.

2.3 Loops and Guardrails

Static skeletons just run once. Systems add **loops** and **guardrails**:

- **Loops** = retry, repair, adapt knobs (e.g., raise `top_k` if faithfulness < 0.7).
- **Guardrails** = validators that block unsafe or unfaithful answers.

Examples:

- *Corrective RAG*: validate → repair (faithfulness < threshold triggers stricter generation).
- *Adaptive RAG*: query looks broad → raise `top_k` and enable reranker; query looks narrow → keep cheap and fast.

2.4 Why this matters

Think of a real-world scenario:

- A medical chatbot that skips guardrails can invent harmful drug advice.
- A financial assistant that ignores adaptive retrieval can miss policy updates.

- A support bot that lacks caching might re-run the same expensive queries hundreds of times per day.

Same skeleton. Different system design. Radically different outcomes.

Knowing loaders, retrievers, and LLMs is like knowing **engine parts**.
Designing a RAG system is like building a **car that drives safely, efficiently, and reliably on actual roads**.

Skeleton = car parts on the garage floor.
System = a tuned, tested vehicle ready for traffic.

This section sets the stage: you don't just want to *use* components, you want to **decide** how to combine them. That's what the next scenarios will teach: when to go corrective, when adaptive, when to loop, and when to cut.

Would you like me to now **draft "Decision Flow diagrams"** (like if Query = broad → Adaptive RAG; if Hallucination risk high → Corrective RAG) so students see a simple *map* of when to choose each system?

3. Sample Decision Flows

3.1 Scenario 1: Corrective RAG (Policy / FAQ Assistant)

Problem

- Domain: internal policies, compliance, finance, healthcare.
- Requirement: Never hallucinate. Safer to refuse than to guess.

When to use: Internal policy, compliance, pricing, warranty—queries where being wrong is worse than being slow.

Flow (high level)

```
User → retrieve(k=8, alpha=0.4, filters on doc_type/policy_version)
     → prepare (MMR, CE re-rank to 8→6, dedup per DocID, light
compression)
     → prompt (strict: "ONLY from CONTEXT, cite [S#], else say 'I
don't know'")
     → generate (low temperature)
```

```
    → VALIDATE:
        1) Faithfulness ≥ 0.8 ? else Repair
        2) Relevancy ≥ 0.8 ? else Reformulate & Retry
    → return answer (or fallback)
```

What adapts and when

- **If Faithfulness < 0.8**
  - Symptom = *Low Faithfulness* (Table 1).
  - Likely Cause = generator fabricating or prompt too loose.
  - Fix Path = **Prompt/Guardrails** (Table 1):
    - Step 1: run a repair pass ("rewrite strictly from CONTEXT"), then re-check.
    - Step 2 (if still < 0.8): widen evidence (**Low Recall** row in Table 1) → raise `top_k` from 8 → 15, relax α from 0.4 → 0.3, regenerate once.
- **If Relevancy < 0.8**
  - Symptom = *Low Answer Relevancy* (Table 1).
  - Likely Cause = prompt misaligned or query misformulated.
  - Fix Path = **Prompt/Retrieve** (Table 1):
    - Keep `k` fixed (avoid Recall inflation), but **reformulate** with multi-query expansion or narrower intent ("focus on taste / refund / eligibility"), then retry once.
- **If both Faithfulness & Relevancy low**
  - Symptom combo = hallucination + off-topic (Table 1: *Low Faithfulness* + *Low Answer Relevancy*).
  - Fix Path = escalate to **safe refusal template** ("I don't know from the provided documents") or escalate to human.

SLOs (tied back to Table 1 metrics)

- **p95 latency ≤ 2.0s** (map to *Latency Spike* row).
- **Hallucination rate < 1%** (monitor *Low Faithfulness* row).
- **Cost/query within fixed budget** (watch for *Cost Spike* row; reranker optional behind flag).

Critical Lines to Wire (cross-check with Table 1 "Telemetry to Log")

- **Feature flags**: `corrective_on`, `use_ce_rerank`, `repair_on`.
- **Tracing**: capture citations, chosen `k`/`α`, validator scores, and final outcome (pass / repair / fallback).

- **Caching**: retrieval cache keyed by normalized query; answer cache only if doc hash unchanged (*Cost Spike* mitigation).

Tiny Controller

```
def corrective_controller(q):
    # 1) retrieve→prepare
    retriever.top_k, retriever.alpha = 8, 0.4
    docs = retriever.get_relevant_documents(q)
    ctx  = format_context(docs)
    ans  = chain.invoke({"history": history, "context": ctx,
"question": q})

    # 2) validators (online)
    f = score_faithfulness(ans, docs)   # Gemini judge
    r = score_relevancy(q, ans)         # Gemini judge

    # 3) corrective loop
    if f < 0.8:
        fixed = repair_with_context(ans, ctx)  # "rewrite strictly
from CONTEXT"
        f2 = score_faithfulness(fixed, docs)
        if f2 >= 0.8: return fixed

        retriever.top_k, retriever.alpha = 15, 0.3
        docs2 = retriever.get_relevant_documents(q)
        ctx2  = format_context(docs2)
        ans2  = chain.invoke({"history": history, "context": ctx2,
"question": q})
        if score_faithfulness(ans2, docs2) >= 0.8: return ans2
        return "I don't know from the provided documents."
    if r < 0.8:
        q2 = reformulate_for_focus(q)   # e.g., "answer only about
taste"
        return corrective_controller(q2)
    return ans
```

3.2 Adaptive RAG for Analyst/Research assistants

When to use: Exploratory, comparative, multi-facet questions; you can trade some latency for deeper coverage.

Flow (high level)

```
User → PLAN (classify breadth & numeracy)
     → retrieve(k, alpha) based on plan
     → prepare (rerank+MMR for broad; minimal prep for numeric)
     → prompt (branch-specific: compare/explain/lookup)
     → generate
     → VALIDATE (faithfulness ≥ 0.7, relevancy ≥ 0.8)
     → If low confidence → widen plan once and retry
```

Planner Heuristics (linked to Table 1)

- **Broad / Comparative queries** ("compare, vs, guide, pros/cons")
  - Risk: *Low Recall* (Table 1) — broad asks miss facets.
  - Fix: raise `k=40`, `fetch_k=60`, keep via CE re-rank=12; enable compression + MMR λ≈0.3–0.4 for diversity; expand context budget.
- **Numeric / Factual queries** (ratios, dates, values)
  - Risk: *Over-Compression* (Table 1) — numbers/units get clipped.
  - Fix: keep `k=20`, CE re-rank=8, **disable LLM compression** for numeric chunks.
- **Default (mixed queries)**
  - Balance between *Low Recall* and *Low Context Utilization* (Table 1).
  - Fix: `k=28`, CE re-rank=10, light compression only.

Offline Tuning (mapped to Table 1 metrics)

- Use **RAGAS Context Recall** to set `k/α` per class (*Low Recall* row).
- Use **Answer Correctness** to decide CE reranker vs pure MMR (*High Recall & Precision, Low Correctness* row).

Online Gates (adaptive triggers)

- **If Faithfulness < 0.7** or model returns "I don't know":
  - Symptom = *Low Faithfulness* (Table 1).

    - Action: widen evidence — increase `k`, relax filters, disable compression — then retry once.
- **If Relevancy < 0.8**:
    - Symptom = *Low Answer Relevancy* (Table 1).
    - Action: query decomposition (multi-query, sub-questions), retry once.

SLOs

- **p95 latency ≤ 3.0s** (budget from *Latency Spike* row; one retry allowed).
- **Recall@K ≥ 0.85 (broad) / ≥ 0.75 (numeric)** (guard against *Low Recall*).
- **Cost/query soft cap**; alert if CE + compression + retry triggered (mitigate *Cost Spike*).

> *This way the students can reason:*
>
> - *Broad query? Risk = Low Recall → fix with ↑k + compression.*
> - *Numeric query? Risk = Over-Compression → keep raw chunks, no summarizer.*
> - *Faithfulness drop? That's Table 1's hallucination row → widen evidence.*

Mini planner + loop

```
def plan(q):
    ql = q.lower()
    broad  = any(w in ql for w in
["compare","vs","guide","pros","cons","steps"])
    numeric= any(ch.isdigit() for ch in ql) or ":" in ql
    if broad:   return
{"k":40,"alpha":0.35,"use_mmr":True,"compress":True}
    if numeric: return
{"k":20,"alpha":0.40,"use_mmr":False,"compress":False}
    return {"k":28,"alpha":0.35,"use_mmr":True,"compress":True}

def adaptive_controller(q):
    p = plan(q)
    retriever.top_k, retriever.alpha = p["k"], p["alpha"]
    docs = retriever.get_relevant_documents(q)
```

```
    ctx  = format_context(docs)
    ans  = chain.invoke({"history": history, "context": ctx,
"question": q})
    f, r = score_faithfulness(ans, docs), score_relevancy(q, ans)
    if f < 0.7 or "I don't know" in ans:
        retriever.top_k += 16
        docs2 = retriever.get_relevant_documents(q)
        ctx2  = format_context(docs2)
        ans2  = chain.invoke({"history": history, "context": ctx2,
"question": q})
        return ans2
    if r < 0.8:
        q2 = decompose_or_multiquery(q)
        return adaptive_controller(q2)
    return ans
```

3.3 Memory-RAG for personal assistants (user prefs + docs)

When to use: Conversational assistants where **user facts** matter (diet, preferences, past choices), but answers must still be grounded in docs.

Flow (high level)

```
User → dual-retrieve:
        A) Docs (k=24, alpha=0.35)
        B) Memory (k=5 from mem_{user_id})
     → prepare (cap max 1-2 memory facts; avoid memory overpowering
docs)
     → prompt (separate blocks: CONTEXT vs MEMORY; require citations
only for CONTEXT)
     → generate
     → VALIDATE:
          Faithfulness to CONTEXT only (do not force citations on
memory)
          Relevancy to user's question (and optionally to memory
signal)
     → If conflict between memory & docs → prefer docs; add hedge
```

Guardrails specific to memory

- Keep **MEMORY** small (top-3 facts) and **clearly separated** in the prompt to avoid "memory as source of truth".
- If memory contradicts docs → answer **from docs**, add a **hedged note** ("You prefer X, but docs say …").
- Do not require citations for memory sentences; require for doc-backed claims.

SLOs

- **Relevancy uplift** (vs doc-only) on personalization queries.
- **Faithfulness** remains ≥ 0.75 to docs.
- **Latency** budget similar to adaptive.

Minimal dual-retrieval controller

```
def memory_rag(q, user_id="user_001"):
    # A) docs
    retriever.top_k, retriever.alpha = 24, 0.35
    docs = retriever.get_relevant_documents(q)

    # B) memory (separate namespace or store)
    mem_docs = mem_retriever(user_id=user_id, k=5)  # your small
memory index
    mem_block = "\n".join(f"- {m.page_content}" for m in
mem_docs[:3]) if mem_docs else "(none)"

    # prompt with separate MEMORY
    ctx = format_context(docs)
    ans = chain_with_memory.invoke({
        "history": history,
        "context": ctx,
        "memory": mem_block,
        "question": q,
    })

    # validate only against CONTEXT
    if score_faithfulness(ans, docs) < 0.75:
```

```
        return repair_with_context(ans, ctx)
    return ans
```

3.4 Short Exercise

Scenario: "Trip-planner for first-time visitors to Kyoto."
Students must design an end-to-end RAG **decision flow** and present:

1. **Choose RAG style(s)**
   - When to go **adaptive** (broad itineraries vs direct facts),
   - When to go **corrective** (visa rules, safety notices).
2. **Define knobs + thresholds**
   - `k`, `alpha`, rerank on/off, compression on/off, pass bars:
     Faithfulness ≥ 0.7, **Relevancy ≥ 0.85**; refusal template for low-support claims.
3. **Sketch the loop**
   - If faithfulness low → widen evidence; if relevancy low → reformulate or decompose; if both low → safe fallback.
4. **SLOs**
   - p95 latency ≤ 2.8s, cost/query cap, nightly RAGAS recall/correctness regression on 50 QAs.
5. **Telemetry plan**
   - Log citations, chosen k/alpha, validator scores, cache hits, and "repair/ retry/ fallback" outcomes.

Deliverable: one diagram (flow from query → plan → retrieve → prepare → prompt → validate → loop), one page of justification (why those thresholds and knobs), and one compact controller function (20–30 lines) implementing their loop.

Scaling RAGs

You've built working RAGs. Now you need to **scale them as products**—with consistent latency, predictable cost, and strong reliability. Below are the **architecture patterns & knobs** you'll reach for most often, explained in plain language and tied back to a relatable example.

Running example: imagine your "Coffee Assistant" has grown into a real product used by two audiences:

- **Consumers** asking taste/recipe questions (broad, fuzzy).
- **Baristas & ops** asking precise, operational questions (ratios, temperatures, SKUs).

We'll keep referring to these two modes to make concrete decisions.

1. Sharding / Partitioning

1.1 What it is (and why you ever bother)

When your knowledge base is small, you can throw every query at one big index and it's fine. As the corpus grows (millions of chunks, many teams, multiple countries, constant updates), two things die first:

1. **Latency** (p95 creeps up because you're searching too much stuff), and
2. **Relevance** (you drag in vaguely-related chatter instead of the *few* passages that matter).

Sharding/partitioning means you **split the corpus into meaningful slices**—and at query time you **search only a small, relevant slice**. Done right, it cuts latency, keeps tokens down, and improves what the LLM sees.

In vector DBs (like Pinecone), you generally partition at the **application level** using:

- **Namespaces** (lightweight "buckets" inside one index), or
- **Separate indices** (heavier isolation, can have different dims/models/settings).

Internally, the vendor might also shard data for you (distribution across pods/partitions), but that's transparent. Here we're talking about **logical** partitioning *you* control.

1.2 The three practical ways to shard

a. Tenant-wise (who the question is for)

Idea: each customer / team / store gets its own "bucket."

When it helps: multi-tenant apps, franchises, departments. One tenant's SOPs shouldn't leak into another's answers.

Coffee Assistant example:
You serve three café chains: **Bean&Co**, **Cupverse**, **RoastLabs**. Each has its own recipes, promos, policy docs. Create a **namespace per tenant**:

- `tenant=beanandco`
- `tenant=cupverse`
- `tenant=roastlabs`

What happens at query time:
A user from Bean&Co asks, "best milk option for saffron latte?"
You **don't** search the global ocean. You search **namespace=beanandco**:

- Less noise → faster retrieval.
- No cross-tenant leakage → safer answers.

How to wire it (Pinecone + LangChain-ish):

```
# upsert with metadata
vectorstore.add_texts(
  texts=chunks,
  metadatas=[{"tenant":"beanandco", "topic":"recipes", "ts":
1724800000}],
  namespace="beanandco"
)

# query-time filter (belt & suspenders)
retriever = vectorstore.as_retriever(
  search_kwargs={"k": 8, "filter": {"tenant": "beanandco"}},
  # plus you can set namespace="beanandco" if you want *hard*
isolation
)
```

Knobs & cautions:

- If tenants are *tiny*, a hard namespace per tenant can produce **empty hits**; combine with a **fallback** to a shared "global" namespace for generic material (e.g., general coffee FAQs).
- If tenants are **huge**, keep tenant namespace but also shard by **topic** inside it (see next).

What to monitor: empty-hit rate per tenant, retrieval p95 per tenant, % of queries that needed the "global fallback."

b. Topic-wise (what the question is about)

Idea: separate by **subject area** (recipes vs. equipment vs. HR vs. policy), because users rarely want all topics at once.

When it helps: mixed corpora where some questions are "taste/compare/benefits" (marketing/blog/recipes) and others are "numbers/ratios/°C/grams" (SOP/equipment).

Coffee Assistant example:
Create topic buckets:

- `topic=recipes` (taste notes, ingredients, steps—consumer-facing)
- `topic=equipment` (ratios, brew temps, grinder settings—ops)
- `topic=policy` (allergens, refunds—support)
- `topic=training` (barista checklists—internal)

You can implement topics either as **namespaces** or as a **metadata field** inside one index:

```
# metadata on each chunk
{"topic":"equipment", "doc_id":"sop_ashwa_001", "heading":"espresso
ratios"}
```

Router at runtime:
Before retrieval, do a **tiny intent classification**:

- Query has **numbers/units** (`1:2`, `93 °C`, grams, ml) → **equipment**.
- Query has **taste words** ("earthy", "bitter", "floral", "compare") → **recipes**.
- Query has **policy words** ("refund", "allergen") → **policy**.

Then you **restrict retrieval**:

```
search_kwargs = {
  "k": 12,
  "filter": {"topic": "recipes"}  # or "equipment"
}
docs =
vectorstore.as_retriever(search_kwargs=search_kwargs).get_relevant_do
cuments(q)
```

What happens:

- Faster: you search fewer chunks.

- Better: you avoid irrelevant domains (no grinder manual when user asked about taste).

Knobs & cautions:

- Keep a **catch-all** (no topic filter) if the router confidence is low.
- Don't explode topics into 20 tiny buckets—2–5 is a sweet spot.
- If a query spans topics (e.g., "taste and brew temp"), you can **query two topics in parallel**, merge, dedup, then rerank.

What to monitor: router confidence, "wrong-topic" rate (sessions where the answer cites sources from a topic different from the router's pick), and token spend (topic filters should reduce it).

c. Time-wise (freshness)

Idea: separate **newly updated** content into a "hot" tier that you always check first.

When it helps: fast-changing docs (menus, promos, seasonal recipes). Users expect "today's" answer.

Coffee Assistant example:

- `tier=hot` → last 90 days
- `tier=warm` → last year
- `tier=cold` → archive

At query time:

1. Search `tier=hot` (and the right topic),
2. If empty, expand to `warm`,
3. If still empty, include `cold`.

You can enforce this with **filters** or **recency boosts**. Filters make *hard* guarantees ("must be recent"), boosts are softer ("prefer recent").

hard filter to hot first

```
filter_hot = {"ts": {"$gte": cutoff_ts}, "topic": "recipes"}
hits = retriever_hot.get_relevant_documents(q)
```

```
# fallback if needed:
filter_all = {"topic": "recipes"}
hits = retriever_all.get_relevant_documents(q)
```

Knobs & cautions:

- Choose a **hot TTL** (60–90 days is common).
- Write new docs **into hot immediately** (streaming ingestion).
- If you re-embed with a different model, do **blue-green**: build a new index in parallel; cut over when warmed and validated.

What to monitor: hit rate in hot tier, fraction of "freshness-intent" queries (those containing "today/this week/new") that got hot hits, staleness complaints.

Strategy	Tenant (multi vs single)	Topic (per topic vs shared)	Time (freshness / TTL)
Namespace (shared index, logical separation)	Pick if you have many small tenants; overhead of separate indexes is too high.	Pick if topics are lightweight or overlapping and benefit from shared recall.	Works if TTL is moderate (60–90 days); easier to re-embed in one sweep. Trade-off: harder to isolate freshness complaints.
Separate Index (physically distinct)	Pick if tenants are large or high-value and isolation is required (billing, noisy neighbor risk).	Pick if topics are very different and embedding space overlaps poorly.	Pick if freshness sensitivity is critical. Allows blue-green cutover per index, at cost of more infrastructure and validation overhead.

2. How routing works (so it doesn't feel magical)

Think of a **two-step router**:

1. **Intent classifier** → what kind of question is this? (consumer-taste / barista-ops / policy / unknown).
    - Simple rules work: look for numbers + units; whitelist taste adjectives; policy keywords.
    - Or use a tiny LLM prompt that returns a class label.

2. **Shard selector** → based on intent, pick:
    - **Tenant** (`namespace=user.tenant_id`)
    - **Topic** filter (`topic="equipment"`)
    - **Time** tier (hot first)

```
intent = route_intent(q)  # "consumer" | "ops" | "policy" | "unknown"
tenant_ns = user.tenant_id or "public"

filters = {}
if intent == "consumer":
    filters["topic"] = "recipes"
    alpha = 0.3   # semantic-leaning
elif intent == "ops":
    filters["topic"] = "equipment"
    alpha = 0.7   # keyword-leaning (preserve numbers)
# time tier: try hot → then relax
filters["ts"] = { "$gte": hot_cutoff }

hits = hybrid_search(ns=tenant_ns, filters=filters, k_dense=18,
k_sparse=24, alpha=alpha)

if not hits:  # fallback ladder
    filters.pop("ts", None)              # drop hot-only
    hits = hybrid_search(ns=tenant_ns, filters=filters, k_dense=24,
k_sparse=32, alpha=alpha)
    if not hits:
        hits = hybrid_search(ns="global", filters={}, k_dense=24,
k_sparse=32, alpha=0.5)
```

Why this matters: it's predictable. When students debug a miss, they can see exactly which gate (intent? tenant? topic? time?) filtered out the answer and fix it.

2.1 What actually changes for your LLM

Nothing mystical. Sharding just **changes the candidate pool** before reranking/compression/prompting. The LLM **sees better, smaller, fresher context**, so:

- **Faithfulness** goes up (no off-domain junk to tempt it),

- **Answer relevancy** goes up (evidence fits the question),
- **Latency & cost** go down (less text to pack & score).

3. Concrete scenarios to cement it

3.1 Scenario 1 — Consumer taste question

"**How does saffron coffee taste compared to turmeric?**"

- **Intent:** consumer → **topic=recipes**, alpha≈0.3 (semantic)
- **Tenant:** user is Bean&Co staff → **namespace=beanandco**
- **Time:** check **hot** first (promos/latest blog), then warm
- **Result:** You retrieve tasting notes from *recipes* docs only, not SOPs. Rerank, compress to taste sentences, answer in 3–4 lines with [S1][S2]. Fast, focused.

3.2 Scenario 2 — Ops numeric question

"**What's the 1:2 ratio and ideal brew temp for the Ashwagandha latte?**"

- **Intent:** ops → **topic=equipment**, alpha≈0.7 (keyword)
- **Tenant:** Bean&Co namespace
- **Time:** hot first; *no compression* (keep numbers!)
- **Result:** You retrieve SOP chunks with exact ratio and °C. Rerank to top 8 (no aggressive summarization). Answer cites the SOP section. No hallucinated numbers.

3.3 Scenario 3 — Freshness mis

"**What's in the new autumn latte released this week?**"

- **Intent:** consumer; **freshness cues** ("new", "this week")
- **Time:** hot filter → empty; fallback to warm → empty
- **Stale-index fallback:** hit CMS/API directly, display answer with "Updated today," then **write-through** that doc into the hot tier so it's searchable on the next query.

3.4 Common faceplants (so you can avoid them)

- **Over-sharding**: 15 tiny topic buckets → you spend all your time hopping and merging. Keep it to a handful that are *obviously different.*

- **Messy metadata**: half your SOPs missing `topic=equipment` → router sends ops queries to recipes and you "mysteriously" miss numbers. Fix your ingestion/enrichment.
- **Compression on numeric content**: extractive summarizers often drop units/ratios. Whitelist doc types where compression = off.
- **No fallback ladder**: strict filters + no gradual relaxation = empty results and angry users. Always have a calm, deterministic ladder (relax time → relax topic → switch to global).
- **No observability**: if you don't log `intent, tenant, topic, tier, k, α, #candidates, rerank_ms`, you won't know which gate is hurting p95 or quality.

4. Throughput and Latency

4.1 Batching (do more work per "turn" without hurting p95)

What "batching" actually means here

- **Embedding batch**: push many chunks through the embedder per call (ingestion time).
- **Parallel retrieval**: run **multiple** retrievals (e.g., topic hot + topic warm) concurrently.
- **Parallel LLM micro-calls**: when you do multi-query, self-query, or verification, **fan out** those LLM calls and **fan in** the results.
- **Streaming answers**: begin returning tokens to the user **while** other background steps finish (e.g., post-logging, analytics).

Batching is mostly an **ingestion/eval** win; for **online queries** you use **parallelism + streaming** to shave p95.

Coffee Assistant: where batching lives

- **Ingestion**: embedding 100k recipe chunks → batch=128 (GPU) or 32 (CPU) to saturate hardware.
- **Online**: run **two shards in parallel** (e.g., `topic=recipes` hot + warm) and merge; don't wait serially.

Minimal patterns

a. Embedding batches at ingest

```
# pseudo: chunk -> embed -> upsert in batches
BATCH = 128
for i in range(0, len(chunks), BATCH):
    batch = chunks[i:i+BATCH]
```

```
    vecs  = embeddings.embed_documents([c.page_content for c in
batch])  # batched under the hood
    index.upsert([
        (c.metadata["chunk_id"], vec, c.metadata)
        for c, vec in zip(batch, vecs)
    ])
```

b. Streaming answer while you log

Let the user see tokens immediately; shove telemetry into a background task. This doesn't lower *compute* but reduces **perceived latency**.

```
runnable = prompt | llm  # no parser while streaming
buf = []
async for chunk in runnable.astream({"question": q, "context":
ctx_str, "history": history}):
    print(chunk.content, end="", flush=True)  # user sees it now
# fire-and-forget: log(buf_text) in a background job
```

Over-parallelization can *increase* p95 (contention + rate limits). Cap concurrency (e.g., 4–8).

Batch too big on embeddings → OOM or throttling. Start small, watch GPU/CPU util, ramp.

Let's try to see what are the ways you can log:

a. Token-by-token logging (inline)

- **How it works:**
  Every time the model produces a token ("Large", "Language", …), your code immediately writes it to a log file, database, or metrics system.
- **Problems:**
  - **Extra overhead:** Each log call might be a DB insert or network request → very slow if repeated hundreds of times.
  - **User delay:** Because logging is in the same loop as token streaming, the user sees stuttery or delayed output.
  - **Resource waste:** Writing 500 small records instead of 1 larger one increases I/O and storage load.
  - **Complex coordination:** Harder to manage retries, batching, or errors in the middle of streaming.

b. Logging the whole text at once

- **How it works:**
  The model finishes generating the full answer, then you log it in a single step.

- **Benefits:**
  - **Efficient I/O:** One big write instead of many small ones.
  - **Simpler code:** No per-token logging logic.

- **Problems:**
  - **Lost detail:** You don't know timing/latency per token.
  - **No early telemetry:** You can't evaluate or monitor the answer until it's completely done.
  - **User experience still blocked:** If logging and evals are synchronous, the user waits until *after* logging to see the answer.

c. The middle ground (the solution)

- Stream tokens to the user immediately.
- Buffer tokens in memory.
- Send them to telemetry (logging, evals, traces) asynchronously in batches.

d. This way:

- The user gets fast, continuous output.
- You still log everything (with timestamps if needed).
- Logging overhead doesn't slow down streaming.

e. In short:

- **Logging every token inline** = accurate but slow, hurts user experience.
- **Logging all text at once** = faster, but you lose granularity and can't monitor streaming behavior.
- **Async/batched logging** = best of both worlds.

```
import asyncio, time, json
from typing import AsyncIterator, Dict, Any

# ---- Fake LLM streamer (replace with real astream) ----
async def fake_llm_stream(prompt: str) -> AsyncIterator[str]:
```

```
    tokens = ["This ", "is ", "streamed ", "output ", "now.\n"]
    for t in tokens:
        await asyncio.sleep(0.15)  # simulate model latency
        yield t

# ---- Telemetry queue & consumer ----
TELEMETRY_Q: asyncio.Queue[Dict[str, Any]] = asyncio.Queue()

async def telemetry_consumer(stop_evt: asyncio.Event):
    batch = []
    last_flush = time.time()
    FLUSH_EVERY_S = 1.5
    BATCH_SIZE = 32
    while not (stop_evt.is_set() and TELEMETRY_Q.empty()):
        try:
            item = await asyncio.wait_for(TELEMETRY_Q.get(),
timeout=0.25)
            batch.append(item)
        except asyncio.TimeoutError:
            pass

        if len(batch) >= BATCH_SIZE or (batch and time.time() -
last_flush > FLUSH_EVERY_S):
            # fire-and-forget write: replace this with your trace
sink (DB, Kafka, S3, OpenTelemetry, etc.)
            print("[telemetry] flush", json.dumps(batch))
            batch.clear()
            last_flush = time.time()

    if batch:
        print("[telemetry] final flush", json.dumps(batch))

# ---- Orchestrator: stream to user, enqueue telemetry ----
async def stream_answer(question: str, context: str, history: str):
    stop_evt = asyncio.Event()
    consumer_task = asyncio.create_task(telemetry_consumer(stop_evt))

    # user stream
```

```
    buf = []
    async for chunk in
fake_llm_stream(prompt=f"Q:{question}\nCTX:{context}\nHIST:{history}"
):
        print(chunk, end="", flush=True)              # user sees it
immediately
        buf.append(chunk)
        await TELEMETRY_Q.put({"t": time.time(), "event": "token",
"chunk": chunk})

    # enqueue final summary + any eval hooks
    buf_text = "".join(buf)
    await TELEMETRY_Q.put({"t": time.time(), "event": "final", "len":
len(buf_text)})
    await TELEMETRY_Q.put({"t": time.time(), "event": "eval_req",
"kind": "hallucination_check"})

    # wind down background
    stop_evt.set()
    await consumer_task

asyncio.run(stream_answer("What is Async?", "Explain streaming vs
background jobs.", ""))
```

4.2 Parameter tuning (the knobs that actually move p95 & quality)

You already know these knobs, but here's how they hit **latency** and **throughput** in practice:

Retrieval knobs

- **k (initial candidates)**: Larger k = better recall, **slower** vector search and heavier downstream re-rank. Typical: **k0 = 24–48** for consumer Qs, **k0 = 16–32** for numeric SOPs.

- `alpha` **(hybrid dense/sparse)**: Closer to **1.0** (sparse) favors exact tokens (numbers, units) and often speeds recall when your sparse index is memory-resident; closer to **0.0** (dense) costs similar but can **increase** re-rank work if it drags in semantically broad candidates. Heuristic:
  - **taste / compare** → `alpha ≈ 0.3`
  - **ratios / temps** → `alpha ≈ 0.7`
- **Topic/time filters**: The best latency win is **less to search**. If the router is confident, **filter hard** (topic, tenant, hot).

Reranking & diversity

- **MMR (`lambda_mult`, `fetch_k`)**: MMR incurs a bit of math but no model inference; it **can** *reduce* downstream tokens (fewer near-dups), which often nets out faster.
- **Cross-encoder**: Huge precision win; adds ms proportional to `k0`. Use it **surgically** (see 2.3).

Prompt & generation

- **Context budget**: The biggest hidden cost. 2k tokens vs 8k tokens is often the difference between sub-second and multi-second. Use **compression & dedup** to cap context (e.g., **≤1200 tokens**).
- **Temperature**: Lower temp trims retries; won't materially change latency, but improves **stability** (fewer self-repairs).
- **Max output tokens**: If you don't cap, users will pay with both **latency** and **$**. "3–6 sentences" works for Q&A.

Quick "knob set" per intent (Coffee)

- **Consumer taste**: `k0=40`, `alpha=0.3`, **CE re-rank** on, **compression** on, budget ≈ 1.2k tokens.
- **Ops numeric**: `k0=24`, `alpha=0.7`, **CE re-rank** on, **compression off** (preserve numbers), budget ≈ 900 tokens.
- **Policy**: `k0=24`, `alpha=0.5`, light compression, strong guardrails.

4.3 Cost controls (you can't scale what you can't afford)

Your real bottleneck is **LLM tokens**. Everything else is peanuts in comparison. Put hard rails in.

a. Budget the context (before the LLM)

- **Dedup** near-identical chunks (semantic or hash).
- **Max 1–2 chunks per source** unless it's a numeric SOP answer.
- **Compression** for narrative content; **no compression** for numeric/specs.
- **Greedy pack** until a fixed token cap (e.g., **1200 tokens**). Anything beyond waits for a retry path.

```
def approx_tokens(text: str) -> int:
    return max(1, len(text)//4)  # rough but consistent

def cap_docs(docs, max_tokens=1200):
    used, kept = 0, []
    for d in docs:
        t = approx_tokens(d.page_content or "")
        if used + t > max_tokens: break
        kept.append(d); used += t
    return kept
```

b. Use the smallest model for side-tasks

- **Faithfulness / relevancy judges**: use a **smaller/cheaper** model (Gemini Flash or similar).
- **Query transforms** (multi-query, self-query): also on the smaller model. Save the **big model** for the final answer only.

c. Dynamic knobs by intent (spend where it matters)

- **Consumer comparisons** benefit from more candidates + compression.
- **Numeric SOP** benefits from fewer candidates but precise re-rank and no compression.
- Tie this to your router to **auto-tune per query**.

d. Caching

- **Retrieval cache**: `(tenant, topic, q_norm)` → list of doc IDs.
- **Prompt-to-answer cache**: hash of final prompt → answer (for FAQs).
- **Embedding dedup**: stable hash of chunk text → skip re-embedding.

e. Backpressure & limits

- Per-request **max time** (e.g., 2.5s budget online). If you hit it, **return partial** with a graceful note and log for offline repair.
- Per-user **rate limits** to keep the system healthy under load.

5. Reliability & Fallbacks in RAG Pipelines

In real-world RAG deployments, retrieval doesn't always behave as expected. Queries may return no results, indexes may be outdated, or the model may drift into unsafe responses. Reliability mechanisms ensure the system degrades gracefully instead of failing silently.

We focus on **three pluggables**:

1. **Empty-hit fallback**
2. **Stale index fallback**
3. **Guardrails**

5.1 Empty-Hit Fallback

When the retriever returns zero or irrelevant documents.

Causes

- User asks something outside scope ("What's the weather tomorrow?" in a coffee-doc index).
- Poor query embedding (OOV terms, misspellings).
- Misconfigured retriever (k too small, filters too strict).

Strategies (LangChain Pluggables / Patterns)

- **Default Answer Prompting**
  - If retrieval is empty, call a prompt that says:
    "No supporting documents were found. Please clarify your question."
- **Fallback Retriever Chain**
  - Primary: Dense (Pinecone).
  - Secondary: Sparse (BM25) or Hybrid with looser filters.
  - LangChain supports chaining retrievers (`EnsembleRetriever` or `MultiQueryRetriever`).

- **Query Relaxation**

    - Auto-drop filters (e.g., remove metadata constraints) and re-run.
    - Rewrite query with MultiQueryRetriever for paraphrases.
- **Out-of-Domain Handling**
    - Detect OOD query → route to a generic LLM answer (without context) with a disclaimer.

5.2 Stale Index Fallback

When the vector store is outdated compared to reality (docs added/updated elsewhere).

Causes

- Incremental ingestion not run (new docs missing).
- Docs updated in source (but index not re-embedded).
- Index corruption or drift (embedding model updated, mismatch in dims).

Strategies

- **Index Freshness Check**
    - Attach timestamps in metadata.
    - If query asks for "latest" or mentions dates newer than index, trigger web/API fallback.
- **Dual-Store Design**
    - Hot store = recent docs (last 7–30 days).
    - Cold store = historical docs.
    - LangChain retrievers can be composed (UnionRetriever, Router).
- **Stale Doc Detection**
    - LLM evaluates doc timestamps → discards those outside acceptable freshness.
    - Example: filter `metadata["ts"] >= now-90d`.

- **External Fallback Source**
    - Route to web search retriever (`TavilySearchRetriever`, `DuckDuckGoRetriever`) if index clearly outdated.

6. Guardrails

Prevent unsafe or ungrounded answers even if retrieval "works."

Risks

- Model hallucination (confident but unsupported).
- Policy violations (medical/financial misstatements, toxic language).
- Overlong, verbose, or uncited answers.

6.1 Guardrail Mechanisms

- **Instructional Guardrails (Prompting)**
  - Clear refusal policy in system prompt:
    "Answer only from CONTEXT. If not found, reply: 'I don't know from the provided documents.'"
- **Validation Chains**
  - Add a **Verifier** chain after answer generation:
    - Check citations present (regex `[S\d+]`).
    - Ask a smaller LLM: "List unsupported claims."
- **Safety Filters**
  - LangChain integrates with **GuardrailsAI**, **PydanticOutputParser**, or regex filters.
  - Drop or rewrite unsafe completions before sending to user.
- **Adaptive Retry**
  - If guardrail flags unsupported claims → re-run with stricter prompt or expand retrieval k.
- **Output Budgeting**
  - Post-process to enforce max sentence length, token budget, JSON schema.

6.2 Key LangChain Pluggables for Reliability:

- `EnsembleRetriever`, `MultiQueryRetriever` → empty-hit fallback.
- `SelfQueryRetriever` with metadata filters → query relaxation.
- `ContextualCompressionRetriever` → stale context control.
- `RouterRetriever`, `UnionRetriever` → dual-store (hot + cold).
- `RunnableLambda` + post-processors → guardrails (citation check, regex filters).
- `JsonOutputParser`, `PydanticOutputParser` → enforce output structure.

Conclusion

You now have the full picture. Building RAG pipelines isn't just about wiring **load → retrieve → prepare → prompt → generate**. You've seen that every stage has *knobs*: chunk size, retriever type, k, reranker on/off, prompt strictness, temperature.

But turning knobs blindly is a trap. Retrieval metrics like Recall@K and Precision@K only tell you *half the story*. They measure pipes, not conversations. A system can "retrieve beautifully" and still mislead the user with an eloquent hallucination.

That's where **RAGAS** steps in. It reframes evaluation around the *end-to-end experience*:

- **Offline metrics** (Context Recall, Context Precision, Answer Correctness) let you regression-test with ground truth.
- **Online metrics** (Faithfulness, Answer Relevancy, Context Relevancy, Context Utilization) let you monitor live systems and enforce guardrails in real time.

Together, they form the **dashboard** that shows whether your knobs are working.

And evaluation alone is not enough — systems need **reliability fallbacks**:

- **Empty-hit fallback** keeps the system graceful when nothing is retrieved.
- **Stale index fallback** ensures freshness by routing to hot tiers or external APIs.
- **Guardrails** prevent unsafe, ungrounded, or policy-violating answers from leaking to users.

In other words:

- **Knobs** = what you've already mastered (retrievers, rerankers, prompts).
- **Dashboard** = RAGAS metrics that show whether you're moving the right dials.
- **Guardrails & Fallbacks** = the safety net that ensures you don't mislead, even when retrieval or generation misbehaves.

A **RAG skeleton** is just components stitched in order. A **RAG system** is what you now know how to build: adaptive, self-checking, and resilient. It knows when to widen search, when to repair, when to admit "I don't know," and when to escalate to a fallback.

That shift — from skeleton to system — is what separates a classroom demo from a production-grade assistant. And RAGAS is the compass that guides you on that path.